DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

All CICD files available in Repo. You can run cicd and view run.

(all code files in repo)

Before You Begin

This ebook includes Terraform configurations, deployment scripts, and other code examples. Although these examples were tested and verified when written, please be aware that:

- **Versions may differ** – Tools and services update over time, which can lead to compatibility issues.
- **AWS services evolve** – Features, APIs, or settings may change after publication.
- **Your setup is unique** – Permissions, account limits, security policies, and network environments vary.
- **Regional factors apply** – Not all AWS services or features are available in every region.

If something doesn't run as expected:
1. Read the error message carefully – it often points directly to the issue.

2. Confirm you're using compatible versions, or try updating to the latest stable release.

3. Consult the current AWS documentation for any service updates.

4. Review your AWS account settings, IAM permissions, and service quotas.

\ Modify and adapt the code to fit your own environment and requirements.

Remember: The purpose of these examples is education, not production-ready copy-paste. Understanding the **why** behind the solution is far more valuable than getting it to run on the first try.

SECTION 7

CI/CD

This section focuses on building automated CI/CD pipelines that handle building, testing, securing, and deploying code without manual steps. Over 2 weeks (12–16 hours), you'll use **GitHub Actions, Docker, kubectl, AWS ECR**, and the security tools from Section 6 all within free tiers.

You'll create pipelines that:

- Build and test code on every commit
- Run automated security checks
- Deploy to multiple environments (dev/staging/prod)
- Support rollbacks and optimizations

By the end, you'll have **production ready CI/CD workflows**, a strong portfolio project, and the ability to clearly explain CI/CD best practices, skills that typically increase salary potential by **$5k–$10k**.

Section 7 Overview

Philosophy:

- Automate everything (build, test, deploy)
- Fail fast, fail early
- Every commit is deployable
- Production patterns only

What you'll build:

- GitHub Actions pipelines
- Automated security scans (from Section 6)
- Multi-environment deployment (dev/staging/prod)
- Rollback capabilities
- Production-grade CI/CD

Tools used:

- GitHub Actions (free tier: 2000 min/month)
- Docker (from Section 3)
- kubectl (from Section 4)
- AWS ECR (from Section 5)
- Security tools (from Section 6)

Section 7 Tasks

Task	Focus	Time	Cost
P1	GitHub Actions Basics	3-4h	Free
P2	Automated Testing & Security	3-4h	Free
P3	Multi-Environment Deployment	4-5h	Free
P4	Pipeline Optimization	2-3h	Free
Total		**12-16h**	**$0**

Cost Breakdown

GitHub Actions:

- Free tier: 2000 minutes/month
- Public repositories: Unlimited minutes
- Private repositories: $0.008/minute after free tier
- Learning usage: Well within the free tier

Total Section 7 cost: $0

Prerequisites

Before starting Section 7, ensure you have:

- Completed Section 6 (D1-D5) - DevSecOps & Security
- Completed Section 5 (T1-T10) - Terraform & AWS
- Completed Section 4 (K1-K9) - Kubernetes
- GitHub account
- AWS account with ECR repository
- EKS cluster running (from Section 5)

Required Tools:

- Git installed
- AWS CLI configured
- kubectl configured
- Docker installed

The Core Concept

CI/CD = Automation for code delivery.

Just as Terraform automates infrastructure and DevSecOps automates security, CI/CD automates the entire software delivery process from commit to production.

Before CI/CD:

- Manual builds
- Manual testing
- Manual deployments
- Human error
- Slow releases

After CI/CD:

- Automated builds on every commit
- Automated testing
- Automated deployments
- Consistent process
- Fast, reliable releases

What You'll Build

Foundation (P1-P2)

- GitHub Actions workflows
- Automated builds and ECR pushes
- Integrated security scans
- Automated testing

Production (P3-P5)

- Multi environment deployments
- Approval gates
- Blue/green deployments
- Pipeline optimization
- Infrastructure CI/CD with Terraform
- Infrastructure security scanning (Checkov)

Success Metrics

By the end of Section 7, you'll have:

- Automated build pipeline (application)
- Automated testing in CI/CD
- Security scans integrated (Trivy, SonarCloud, tfsec, Checkov)
- Multi-environment deployment
- Infrastructure CI/CD (Terraform)
- Rollback capabilities
- Optimized pipeline performance

Career Impact:

- **Resume:** "Implemented CI/CD pipelines with GitHub Actions."
- **Portfolio:** Production-ready workflows
- **Interview-ready:** Can explain CI/CD best practices
- **Salary potential:** $5,000-10,000 increase

Ready to Start?

Next: P1 — GitHub Actions Basics (3-4 hours)

P1: GitHub Actions Basics

Time: 3-4 hours | **Cost:** Free

Mission Brief

Create your first GitHub Actions pipeline that builds Docker images and pushes them to ECR automatically on every commit.

Outcomes:

- Understand CI/CD fundamentals
- Create GitHub Actions workflows
- Configure AWS credentials securely
- Automate Docker builds and ECR pushes
- Set up GitHub secrets

Prerequisites: Section 5 (T1-T10) complete, ECR repository exists

The One Thing That Makes This Click

CI/CD = Assembly line for code. Just like a car factory builds cars automatically when parts arrive, GitHub Actions builds and deploys your app automatically when code is pushed.

Without CI/CD:

- Build manually
- Test manually
- Deploy manually
- Hope nothing breaks

With CI/CD:

- Push code
- Pipeline builds automatically
- Tests run automatically
- Deploys automatically
- Consistent, repeatable process

Phase 1: Understanding (45 min)

What is CI/CD?

CI (Continuous Integration): Automatically build and test code on every commit.

CD (Continuous Deployment): Automatically deploy passing builds to production.

Why GitHub Actions?

- Free tier: 2000 minutes/month
- Built into GitHub
- YAML configuration
- Large marketplace of actions

Key Concepts

- **Workflow:** YAML file defining your pipeline
- **Job:** Group of steps that run on same runner
- **Step:** Individual task (checkout, build, deploy)
- **Runner:** VM that executes your workflow
- **Trigger:** Event that starts workflow (push, PR, schedule)

Design Exercise 1: Pipeline Flow

Draw your pipeline:

- What happens from `git push` to the app being deployed?
- Which steps can fail?
- Where to add quality gates?

Example flow:

```
git push
  ↓
Checkout code
  ↓
Build Docker image
  ↓
Push to ECR
  ↓
Deploy to Kubernetes
```

Design Exercise 2: Choose Triggers

When should the pipeline run?

- Push to main? (Yes - deploy to production)
- Pull requests? (Yes - test before merge)
- Schedule? (Maybe - nightly builds)
- Documentation changes? (No - skip CI)

Which branches deploy where?

- main → production
- staging → staging
- dev → dev

Phase 2: Implementation (2 hours)

Step 0: Ensure ECR Repositories Exist

Check if ECR repositories exist:

```
aws ecr describe-repositories --repository-names h-game-backend
h-game-frontend --region us-east-1
```

If repositories don't exist, create them:

Option A: Via Terraform (Recommended)

```
cd h-game-terraform
terraform apply -target=aws_ecr_repository.backend
-target=aws_ecr_repository.frontend
```

Option B: Manually

```
# Backend repository
aws ecr create-repository \
  --repository-name h-game-backend \
  --region us-east-1 \
  --image-scanning-configuration scanOnPush=true \
  --encryption-configuration encryptionType=AES256

# Frontend repository
aws ecr create-repository \
  --repository-name h-game-frontend \
  --region us-east-1 \
  --image-scanning-configuration scanOnPush=true \
  --encryption-configuration encryptionType=AES256
```

Get ECR registry URL:

```
# Get your AWS account ID
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output
text)

# ECR Registry URL
ECR_REGISTRY="${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com"
echo "ECR Registry: $ECR_REGISTRY"
# Example: 334091769766.dkr.ecr.us-east-1.amazonaws.com
```

Checkpoint: ECR repositories exist

Step 1: Create Complete Workflow

Create `.github/workflows/build.yml` (builds both backend and frontend):

```
name: Build and Push

on:
  push:
    branches: [main, develop]
    paths:
      - 'backend/**'
      - 'frontend/**'
      - '.github/workflows/build.yml'
  pull_request:
    branches: [main]
    paths:
      - 'backend/**'
      - 'frontend/**'
      - '.github/workflows/build.yml'

env:
  AWS_REGION: us-east-1
  ECR_REGISTRY: ${{ secrets.ECR_REGISTRY }}

jobs:
  build-backend:
```

```yaml
    name: Build Backend
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: ${{ env.AWS_REGION }}

    - name: Login to ECR
      uses: aws-actions/amazon-ecr-login@v2

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Build and push backend
      uses: docker/build-push-action@v5
      with:
        context: ./backend
        push: true
        tags: |
          ${{ env.ECR_REGISTRY }}/h-game-backend:${{ github.sha }}
          ${{ env.ECR_REGISTRY }}/h-game-backend:latest

  build-frontend:
    name: Build Frontend
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Configure AWS credentials
```

```
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: ${{ env.AWS_REGION }}

    - name: Login to ECR
      uses: aws-actions/amazon-ecr-login@v2

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Build and push frontend
      uses: docker/build-push-action@v5
      with:
        context: ./frontend
        push: true
        tags: |
          ${{ env.ECR_REGISTRY }}/h-game-frontend:${{ github.sha }}
          ${{ env.ECR_REGISTRY }}/h-game-frontend:latest
```

Note: This workflow builds both backend and frontend in parallel jobs for faster execution.

Step 2: Add GitHub Secrets

1. Navigate to GitHub repo → Settings → Secrets and variables → Actions
2. Click "New repository secret."
3. Add these secrets:
    - `AWS_ACCESS_KEY_ID`: Your AWS access key
    - `AWS_SECRET_ACCESS_KEY`: Your AWS secret key
    - `ECR_REGISTRY`: Your ECR registry URL (e.g., `123456789012.dkr.ecr.us-east-1.amazonaws.com`)

Get ECR registry URL:

```
# Get your AWS account ID
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output
text)
```

```
# ECR Registry URL (format: ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com)
```

Step 5: Test Pipeline

```
git add .github/workflows/build.yml
git commit -m "feat: Add CI/CD pipeline."
git push origin main

# Watch pipeline run in GitHub Actions tab
```

Expected result:

- Workflow appears in the Actions tab
- Build job runs successfully
- Image appears in ECR

P1 Verification Guide

How to Verify Your Build and Push Workflow Worked
Method 1: Check GitHub Actions (Primary)

1. Go to your GitHub repository
    - Navigate to: `https://github.com/YOUR_USERNAME/YOUR_REPO`
2. Click "Actions" tab
    - You should see "Build and Push" workflow
3. Check workflow runs:
    - ✅ Green checkmark = Success
    - ❌ Red X = Failed
    - 🟡 Yellow circle = In progress
4. Click on a workflow run to see:
    - `build-backend` job status
    - `build-frontend` job status
    - Build logs
    - Any errors

Method 2: Check ECR for Images

If workflow succeeded, you should see images in ECR:

```
# Check backend images
aws ecr list-images --repository-name h-game-backend --region
us-east-1

# Check frontend images
aws ecr list-images --repository-name h-game-frontend --region
us-east-1
```

Expected output:

- Images tagged with commit SHA (e.g., `abc123def456...`)
- Images tagged `latest`

Method 3: Check Recent Commits

Verify workflow was triggered:

```
cd "/Users/mac/Desktop/devops playbook"
git log --oneline -5
```

If you see a commit with the workflow file, check:

- Did you push to `main` or `develop` branch?
- Did you modify files in `backend/` or `frontend/` directories?

Current Status Check

Run this to check the current status:

```
# Check if workflow file exists
ls -la .github/workflows/build.yml

# Check if ECR repos have images
aws ecr describe-images --repository-name h-game-backend --region
us-east-1 --max-items 1
```

```
aws ecr describe-images --repository-name h-game-frontend --region
us-east-1 --max-items 1

# Check recent git commits
git log --oneline --all -5
```

If Workflow Hasn't Run Yet

Possible reasons:

1. Workflow file not pushed to GitHub
2. GitHub secrets not configured
3. No commits to `backend/` or `frontend/` directories
4. Pushed to the wrong branch (not `main` or `develop`)

To trigger workflow:

```
# Make a change to backend or frontend
echo "# Test" >> backend/README.md
git add backend/README.md .github/workflows/build.yml
git commit -m "test: Trigger CI/CD workflow"
git push origin main
```

If Workflow Failed

Common issues:

1. Missing GitHub secrets
    - Check: Settings → Secrets → Actions
    - Required: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `ECR_REGISTRY`
2. ECR permissions
    - IAM user needs: `ecr:GetAuthorizationToken`, `ecr:PutImage`, etc.
3. Docker build fails
    - Check build logs in GitHub Actions
    - Test locally: `docker build -t test ./backend`

Success Indicators

- Workflow shows green checkmark in GitHub Actions
- ECR repositories contain images
- Images tagged with commit SHA and `latest`
- Both backend and frontend jobs completed

To verify right now, check your GitHub Actions tab!

Phase 3: Validation (30 min)

Verify Pipeline Success

1. Check GitHub Actions tab (green checkmark)
2. Verify image in ECR:

```
aws ecr list-images --repository-name h-game-backend
```

3. Check workflow logs for errors
4. Test on different branches

Common Issues & Fixes

Issue 1: Missing ECR_REGISTRY Secret

Problem: Workflow fails with "ECR_REGISTRY not found."

Fix: Add `ECR_REGISTRY` to GitHub Secrets

- **Go to:** Settings → Secrets → Actions
- **Add:** `ECR_REGISTRY` = `YOUR_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com`
- **Get account ID:** `aws sts get-caller-identity --query Account --output text`

Issue 2: Multiple Workflows Triggering

Problem: Multiple workflows run on the same push (Build, Terraform, etc.)

Fix: This is normal. Workflows trigger based on file paths:

- `build.yml` → triggers on `backend/**` or `frontend/**` changes
- `terraform.yml` → triggers on `h-game-terraform/**` changes
- Each workflow only runs when relevant files change

Issue 3: ECR Repository Not Found

Problem: `RepositoryNotFoundException` when pushing images

Fix: Create ECR repositories first (see Step 0 above) or run:

```
aws ecr create-repository --repository-name h-game-backend --region
us-east-1
aws ecr create-repository --repository-name h-game-frontend --region
us-east-1
```

Issue 4: Workflow Not Triggering

Problem: Pushed code, butthe workflow didn't run

Fix: Check:

1. Did you push to `main` or `develop` branch? (workflow only runs on these)
2. Did you modify files in `backend/` or `frontend/`? (path filter)
3. Is workflow file in `.github/workflows/` directory?

Issue 5: Docker Build Fails

Problem: Build step fails with Docker errors

Fix:

1. Test locally: `docker build -t test ./backend`
2. Check Dockerfile exists in `backend/` and `frontend/` directories
3. Verify Dockerfile syntax is correct

Issue 6: AWS Credentials Invalid

Problem: Authentication errors

Fix:

- Re-create the access key in AWS IAM

- Verify IAM user has ECR permissions: `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailability`, `ecr:PutImage`

Key Takeaways

- GitHub Actions triggers on git events
- Workflows defined in YAML
- Secrets stored securely in GitHub
- Automated builds reduce manual work

Bridge to P2

Now images build automatically, but how do we know they're safe and working? Let's add testing and security scans...

P2: Automated Testing & Security

Time: 3-4 hours | **Cost:** Free

Mission Brief

Integrate security scans (Trivy, tfsec, SonarCloud from Section 6) and automated tests into your pipeline. Block deployments on failures.

Outcomes:

- Run unit tests in CI/CD
- Integrate Trivy container scanning
- Integrate tfsec infrastructure scanning
- Integrate SonarCloud code quality
- Block deployments on quality gate failures

Prerequisites: P1 complete, Section 6 (D1-D5) complete

The One Thing That Makes This Click

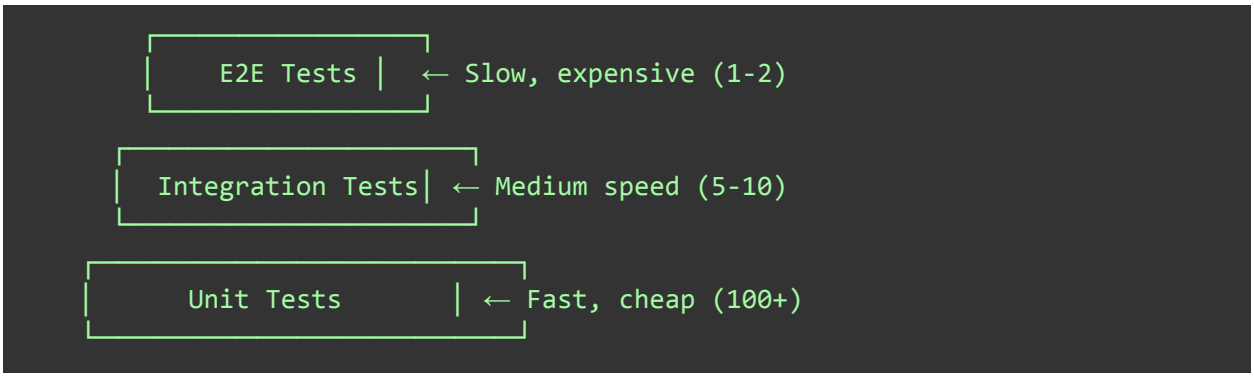
Testing in CI/CD is a quality check before release.
Just like a factory inspects products before shipping, a pipeline checks code before deployment.

Without quality checks, broken or insecure code reaches production, and problems are fixed later.
With quality gates, tests, security scans, and code quality checks must pass first.

Only code that passes all checks is deployed.

Phase 1: Understanding (30 min)

Test Pyramid



Quality Gates

Unit tests: Pass rate > 80%
Security scan: No critical/high CVEs
Code quality: No blocker issues
Infrastructure: No high/critical findings

Design Exercise: Map Your Gates

List all quality checks needed:

- Unit tests
- Container scanning (Trivy)
- Infrastructure scanning (tfsec)
- Code quality (SonarCloud)

Order checks (fail fast):

1. Unit tests (fastest)
2. Security scans (medium)
3. Code quality (slower)

What blocks deployment:

- Critical/high CVEs
- Failed tests
- Blocker code quality issues

Phase 2: Implementation (2.5 hours)

Step 1: Add Unit Tests

Add test job to workflow:

```
test:
   runs-on: ubuntu-latest

   steps:
   - uses: actions/checkout@v4

   - name: Set up Node.js
     uses: actions/setup-node@v4
     with:
       node-version: '18'

   - name: Install dependencies
     run: npm ci
     working-directory: ./backend

   - name: Run unit tests
     run: npm test -- --coverage
     working-directory: ./backend

   - name: Upload coverage
     uses: codecov/codecov-action@v4
     with:
       files: ./backend/coverage/coverage-final.json
```

Step 2: Add Trivy Scan

Add security scan job (runs after build):

```
security-scan:
   needs: build
   runs-on: ubuntu-latest

   steps:
```

```
    - name: Run Trivy scan
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: ${{ secrets.ECR_REGISTRY }}/h-game-backend:${{
github.sha }}
        format: 'sarif'
        output: 'trivy-results.sarif'
        severity: 'CRITICAL,HIGH'
        exit-code: '1'

    - name: Upload results
      uses: github/codeql-action/upload-sarif@v2
      if: always()
      with:
        sarif_file: 'trivy-results.sarif'
```

Step 3: Add SonarCloud

Add code quality job:

```
 code-quality:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0

    - name: SonarCloud Scan
      uses: SonarSource/sonarcloud-github-action@master
      env:
        GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
```

Set up SonarCloud:

1. Sign up at sonarcloud.io (free for open source)
2. Import your GitHub repository
3. Get project key and organization key
4. Generatea token in SonarCloud
5. Add `SONAR_TOKEN` to GitHub secrets

Step 4: Add tfsec Scan

Add infrastructure scan job:

```
 infrastructure-scan:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4

    - name: Run tfsec
      uses: aquasecurity/tfsec-action@v1.0.0
      with:
        soft_fail: false
        severity: HIGH
```

Step 5: Parallelize Jobs

Update workflow to run jobs in parallel:

```
jobs:
  build:
    runs-on: ubuntu-latest
    # ... build steps ...

  test:
    runs-on: ubuntu-latest
    # runs independently

  security-scan:
    needs: build
```

```
    runs-on: ubuntu-latest
    # runs after build

  code-quality:
    runs-on: ubuntu-latest
    # runs independently

  infrastructure-scan:
    runs-on: ubuntu-latest
    # runs independently

  deploy:
    needs: [build, test, security-scan, code-quality,
infrastructure-scan]
    runs-on: ubuntu-latest
    # runs after all pass
```

Phase 3: Validation (30 min)

Test Each Gate

1. Break unit test:

```
// backend/tests/api.test.js
test('should fail', () => {
  expect(1).toBe(2); // This will fail
});
```

2. Add vulnerable dependency:

```
// package.json
"dependencies": {
  "lodash": "4.17.4" // Known vulnerability
}
```

3. Add code smell:

```
// Add duplicate code or complexity
function badFunction() {
  if (true) {
    if (true) {
      if (true) {
        // Too nested
      }
    }
  }
}
```

4. Add insecure Terraform:

```
# terraform/security-group.tf
resource "aws_security_group" "bad" {
  ingress {
    from_port = 0
    to_port = 65535
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"] # Public access
  }
}
```

Verify

- All jobs show in GitHub Actions UI
- Failed jobs show clear error messages
- Successful jobs have green checkmarks
- PR comments show scan results

Key Takeaways

- Multiple quality gates protect production
- Failed gates block deployment
- Security scans from Section 6 are integrated
- Parallel jobs reduce pipeline time

Bridge to P3

Tests pass, security is validated. Now let's deploy to multiple environments safely...

P3: Multi-Environment Deployment

Time: 4-5 hours | **Cost:** Free

Mission Brief

Deploy automatically to dev/staging/prod environments with appropriate approval gates. Implement blue/green deployments for zero-downtime updates.

Outcomes:

- Configure multiple environments
- Set up approval gates
- Implement blue/green deployments
- Add rollback capabilities
- Configure deployment notifications

Prerequisites: P2 complete, multiple EKS clusters or namespaces

The One Thing That Makes This Click

Multi-environment deployment = Dress rehearsal before opening night. Just like plays rehearse before performing, code deploys to dev/staging before production.

Without environments:

- Deploy directly to production
- Break production
- Fix in production
- Users affected

With environments:

- Test in dev first
- Validate in staging
- Deploy to production safely
- Users unaffected

Phase 1: Understanding (45 min)

Environment Strategy

DEV:

- Latest code
- Frequent deployments (every commit)
- Break things safely
- Developers test here

STAGING:

- Stable code
- Production-like environment
- QA testing
- Client demos

PRODUCTION:

- Proven code
- Scheduled deployments
- Real users
- Requires approval

Deployment Strategies

Rolling: Update Pods one by one
Blue/Green: Switch traffic between two identical environments
Canary: Gradually shift traffic to the new version

Design Exercise: Environment Flow

Map branches to environments:

- dev branch → DEV environment
- staging branch → STAGING environment
- main branch → PRODUCTION environment

Decide approval requirements:

- **DEV:** No approval (automatic)
- **STAGING:** 1 reviewer
- **PRODUCTION:** 2 reviewers + wait timer

Plan rollback strategy:

- Automatic rollback on deployment failure
- Manual rollback via GitHub Actions
- Keep the previous version for 10 minutes

Phase 2: Implementation (3 hours)

Step 1: Configure Environments in GitHub

1. Navigate to GitHub repo → Settings → Environments
2. Create three environments:

DEV:

- No protection rules
- Auto deploy

STAGING:

- Required reviewers: 1
- Wait timer: 0 minutes

PRODUCTION:

- Required reviewers: 2
- Wait timer: 5 minutes
- Restrict to main branch only

Cost Saving Alternative:

Instead of 3 separate EKS clusters (~$300/month), you can use 3 namespaces in one cluster (~$100/month):

```
# Create namespaces in your existing cluster
kubectl create namespace h-game-dev
kubectl create namespace h-game-staging
# h-game namespace already exists for (production)
```

Important: All environments use the same EKS cluster (h-game-prod) but different namespaces for cost savings.

Step 2: Environment-Specific Workflow

Create `.github/workflows/deploy.yml`:

```
name: Deploy

on:
  push:
    branches: [dev, staging, main]

jobs:
  determine-environment:
    runs-on: ubuntu-latest
    outputs:
      environment: ${{ steps.set-env.outputs.environment }}
      cluster: ${{ steps.set-env.outputs.cluster }}

    steps:
    - name: Set environment
      id: set-env
      run: |
        if [[ "${{ github.ref }}" == "refs/heads/main" ]]; then
          echo "environment=production" >> $GITHUB_OUTPUT
          echo "cluster=h-game-prod" >> $GITHUB_OUTPUT
        elif [[ "${{ github.ref }}" == "refs/heads/staging" ]]; then
          echo "environment=staging" >> $GITHUB_OUTPUT
```

```
          echo "cluster=h-game-staging" >> $GITHUB_OUTPUT
        else
          echo "environment=dev" >> $GITHUB_OUTPUT
          echo "cluster=h-game-dev" >> $GITHUB_OUTPUT
        fi

  deploy:
    needs: [build, test, security-scan, determine-environment]
    runs-on: ubuntu-latest
    environment: ${{ needs.determine-environment.outputs.environment
}}

    steps:
    - uses: actions/checkout@v4

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: us-east-1

    - name: Update kubeconfig
      run: |
        aws eks update-kubeconfig \
          --name ${{ needs.determine-environment.outputs.cluster }} \
          --region us-east-1

    - name: Deploy to Kubernetes
      run: |
        kubectl set image deployment/backend \
          backend=${{ secrets.ECR_REGISTRY }}/h-game-backend:${{
github.sha }} \
          -n h-game

        kubectl rollout status deployment/backend -n h-game
```

Step 3: Create Blue/Green Manifests

Before implementing blue/green deployment, create the necessary manifests:

Create `k8s/deployment-blue.yml` (namespace will be set at apply time)**:**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-blue
  # namespace will be specified with the -n flag
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
      version: blue
  template:
    metadata:
      labels:
        app: backend
        version: blue
    spec:
      containers:
      - name: backend
        image: PLACEHOLDER_ECR_REGISTRY/h-game-backend:latest
        # Note: Image will be updated by kubectl set image during
deployment
        ports:
        - containerPort: 3001
```

Create `k8s/deployment-green.yml`**:**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-green
  # namespace will be specified with the -n flag
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
      version: green
  template:
    metadata:
      labels:
        app: backend
        version: green
    spec:
      containers:
      - name: backend
        image: PLACEHOLDER_ECR_REGISTRY/h-game-backend:latest
        # Note: Image will be updated by kubectl set image during
deployment
        ports:
        - containerPort: 3001
```

Create `k8s/service-blue.yml`**:**

```
apiVersion: v1
kind: Service
metadata:
  name: backend-blue-service
  # namespace will be specified with -n flag
spec:
  selector:
    app: backend
    version: blue
```

```
  ports:
  - port: 3001
    targetPort: 3001
```

Create `k8s/service-green.yml`**:**

```
apiVersion: v1
kind: Service
metadata:
  name: backend-green-service
  # namespace will be specified with -n flag
spec:
  selector:
    app: backend
    version: green
  ports:
  - port: 3001
    targetPort: 3001
```

Initial setup:

```
# Create namespaces if they don't exist
kubectl create namespace h-game-dev
kubectl create namespace h-game-staging
# h-game namespace already exists (production)

# Deploy blue (current production) to each namespace
for ns in h-game h-game-staging h-game-dev; do
  kubectl apply -f k8s/deployment-blue.yml -n $ns
  kubectl apply -f k8s/service-blue.yml -n $ns

  # Update main service to point to blue
  kubectl patch service backend -n $ns \
    -p '{"spec":{"selector":{"version":"blue"}}}'
done
```

Step 4: Blue/Green Deployment

Add blue/green deployment step:

```
   - name: Blue/Green Deploy
      run: |
        NAMESPACE=${{ needs.determine-environment.outputs.namespace
}}

        # Deploy to green (inactive) environment
        kubectl apply -f k8s/deployment-green.yml -n $NAMESPACE

        # Wait for green to be ready
        kubectl wait --for=condition=available \
          deployment/backend-green -n $NAMESPACE --timeout=300s

        # Run smoke tests on green
        kubectl run smoke-test --image=curlimages/curl \
          --rm -i --restart=Never -n $NAMESPACE -- \
          curl -f
http://backend-green-service.$NAMESPACE.svc.cluster.local:3001/health

        # Switch traffic from blue to green
        kubectl patch service backend -n $NAMESPACE \
          -p '{"spec":{"selector":{"version":"green"}}}'

        # Keep blue running for 10 minutes (rollback window)
        # This is intentional but adds 10 minutes to pipeline
duration
        # Allows quick rollback by switching service selector back to
blue
        sleep 600

        # Scale down blue after rollback window
        kubectl scale deployment/backend-blue --replicas=0 -n
$NAMESPACE
```

Step 5: Rollback Strategy

Add rollback job:

```
 rollback:
    if: failure()
    needs: [deploy, determine-environment]
    runs-on: ubuntu-latest

    steps:
    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: us-east-1

    - name: Rollback deployment
      run: |
        NAMESPACE=${{ needs.determine-environment.outputs.namespace
}}

        aws eks update-kubeconfig --name h-game-prod --region
us-east-1

        # Rollback to previous version
        kubectl rollout undo deployment/backend -n $NAMESPACE

        # Wait for rollback to complete
        kubectl rollout status deployment/backend -n $NAMESPACE

    - name: Notify team
      uses: 8398a7/action-slack@v3
      with:
        status: failure
        text: 'Deployment failed. Automatic rollback initiated.'
        webhook_url: ${{ secrets.SLACK_WEBHOOK }}
```

Blue/Green Deployment and Rollback:

Blue/green deploys two versions at once:
Blue (current production) and green (new version).
Green is deployed and tested while blue serves traffic. After green passes smoke tests, traffic switches to green. Blue stays running for 10 minutes as a rollback window. If issues occur, the rollback job switches the service selector back to blue, instantly reverting traffic to the previous working version.

After the rollback window, blue is scaled down to save resources. This provides zero-downtime deployments with a quick rollback path.

Step 6: Deployment Notifications

Add notification job:

```
 notify:
    if: always()
    needs: deploy
    runs-on: ubuntu-latest

    steps:
    - name: Slack notification
      uses: 8398a7/action-slack@v3
      with:
        status: ${{ job.status }}
        text: |
          Deployment to ${{
needs.determine-environment.outputs.environment }}
          Status: ${{ job.status }}
          Commit: ${{ github.sha }}
          Author: ${{ github.actor }}
        webhook_url: ${{ secrets.SLACK_WEBHOOK }}
```

Note: The `sleep 600` command keeps the pipeline running for 10 minutes to maintain the rollback window. This is intentional but adds 10 minutes to your pipeline duration. If you need faster pipelines, you can reduce this time or use a background job to handle the cleanup.

Phase 3: Validation (45 min)

Test Deployment Flow

Test DEV deployment:

```
# Ensure namespace exists
kubectl create namespace h-game-dev

git checkout dev
git commit -m "test: DEV deployment"
```

```
git push origin dev
# Should deploy automatically to h-game-dev namespace
```

Test STAGING deployment:

```
# Ensure namespace exists
kubectl create namespace h-game-staging

git checkout staging
git merge dev
git push origin staging
# Should require 1 approval, then deploy to h-game-staging namespace
```

Test PROD deployment:

```
# h-game namespace should already exist
git checkout main
git merge staging
git push origin main
# Should require 2 approvals + 5 min wait, then deploy to h-game
namespace
```

Verify

- DEV deploys without approval to `h-game-dev` namespace
- STAGING requires 1 approval, deploys to `h-game-staging` namespace
- PROD requires 2 approvals + wait, deploys to `h-game` namespace
- All environments use the same cluster (`h-game-prod`)
- Rollback works on failure
- Notifications sent to Slack

Verify deployments:

```
# Check all namespaces
kubectl get deployments -n h-game-dev
kubectl get deployments -n h-game-staging
kubectl get deployments -n h-game
```

Key Takeaways

- Multiple environments protect production
- Approval gates prevent accidental deploys
- Blue/green enables zero-downtime
- Rollback automated on failure

Bridge to P4

Deployments work, but pipelines are slow and wasteful. Let's optimize...

Verify

- DEV deploys without approval
- STAGING requires 1 approval
- PROD requires 2 approvals + wait
- Rollback works on failure
- Notifications sent to Slack

P4: Pipeline Optimization & Best Practices

Time: 2-3 hours | **Cost:** Free

Mission Brief

Optimize pipeline speed with caching, matrix builds, and reusable workflows. Implement monitoring and cost controls.

Outcomes:

- Implement dependency caching
- Use matrix builds for parallel testing
- Create reusable workflows
- Add conditional execution
- Monitor pipeline performance

Prerequisites: P3 complete

The One Thing That Makes This Click

Pipeline optimization = Efficient factory. Just like factories optimize for speed and cost, pipelines should run fast without wasting resources.

Before optimization:

- Pipeline takes 15 minutes
- Uses 30 CI minutes
- Rebuilds everything each time
- Slow feedback

After optimization:

- Pipeline takes 5 minutes
- Uses 10 CI minutes
- Reuses cached dependencies
- Fast feedback

Phase 1: Understanding (30 min)

Optimization Strategies

Caching: Reuse dependencies between runs
Matrix builds: Test multiple versions in parallel
Conditional execution: Skip unnecessary steps
Reusable workflows: DRY principle for pipelines

Cost Management

GitHub Actions:

- Free tier: 2000 min/month
- After that: $0.008/minute
- Optimization saves money and time

Design Exercise: Identify Bottlenecks

Review current pipeline:

- Which job takes the longest?
- What dependencies are rebuilt?
- Which steps can run in parallel?
- What can be skipped?

Example bottlenecks:

- npm install (2 min) → Cache node_modules
- Docker build (5 min) → Cache Docker layers
- Sequential jobs → Run in parallel

Phase 2: Implementation (2 hours)

Step 1: Add Dependency Caching

Add caching to the build job:

```
 build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4

    - name: Cache Node modules
      uses: actions/cache@v3
      with:
        path: ~/.npm
        key: ${{ runner.os }}-node-${{
hashFiles('**/package-lock.json') }}
        restore-keys: |
          ${{ runner.os }}-node-

    - name: Cache Docker layers
      uses: actions/cache@v3
      with:
        path: /tmp/.buildx-cache
        key: ${{ runner.os }}-buildx-${{ github.sha }}
        restore-keys: |
          ${{ runner.os }}-buildx-
```

Step 2: Matrix Builds

Add matrix strategy to test job:

```
 test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
```

```
        node-version: [16, 18, 20]
        os: [ubuntu-latest, macos-latest]
      fail-fast: false

    steps:
    - uses: actions/checkout@v4

    - name: Use Node.js ${{ matrix.node-version }}
      uses: actions/setup-node@v4
      with:
        node-version: ${{ matrix.node-version }}

    - name: Run tests
      run: npm test
```

Step 3: Conditional Execution

Add conditions to skip unnecessary steps:

```
deploy:
    if: |
      !contains(github.event.head_commit.message, '[skip ci]') &&
      !contains(github.event.head_commit.message, 'docs:')

    steps:
    - name: Deploy only on main
      if: github.ref == 'refs/heads/main'
      run: ./deploy.sh
```

Step 4: Reusable Workflows

Create `.github/workflows/reusable-build.yml`:

```yaml
name: Reusable Build

on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
    secrets:
      AWS_ACCESS_KEY_ID:
        required: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - name: Build for ${{ inputs.environment }}
      run: docker build -t app:${{ inputs.environment }} .
```

Use in main workflow:

```yaml
# .github/workflows/deploy-all.yml
name: Deploy All Environments

on: push

jobs:
  deploy-dev:
    uses: ./.github/workflows/reusable-build.yml
    with:
      environment: dev
    secrets:
      AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
```

```
 deploy-staging:
   uses: ./.github/workflows/reusable-build.yml
   with:
     environment: staging
   secrets:
     AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
```

Step 5: Pipeline Monitoring

Add monitoring job:

```
monitor:
   runs-on: ubuntu-latest
   if: always()

   steps:
   - name: Track pipeline metrics
     run: |
       echo "Duration: ${{ job.duration }}"
       echo "Status: ${{ job.status }}"

   - name: Check GitHub Actions usage
     run: |
       gh api /repos/${{ github.repository }}/actions/runs \
         --jq '.total_count'
```

Step 6: Cost Optimization Rules

Add conditional execution for expensive tests:

```
expensive-tests:
   if: |
     github.ref == 'refs/heads/main' ||
     github.event_name == 'pull_request'
   runs-on: ubuntu-latest
```

```
    steps:
    - name: E2E tests
      run: npm run test:e2e
```

Phase 3: Validation (30 min)

Measure Improvements

Before optimization:

- Pipeline duration: 15 minutes
- CI minutes used: 30 minutes
- Cost: $0.24 per run (if over free tier)

After optimization:

- Pipeline duration: 5 minutes
- CI minutes used: 10 minutes
- Cost: $0.08 per run (if over free tier)

Improvement: 66% faster, 66% cheaper

Verify

- Cache hits reduce build time
- Matrix builds run in parallel
- Documentation changes skip CI
- Reusable workflows eliminate duplication
- Monthly usage tracked

Key Takeaways

- Caching saves 2-5 minutes per run
- Parallel jobs reduce total time
- Conditional execution saves CI minutes
- Reusable workflows = DRY for pipelines

Section 7 Summary

What You Built:

- P1: GitHub Actions pipelines (build + push to ECR)
- P2: Automated testing + security scans (quality gates)
- P3: Multi-environment deployment (dev/staging/prod)
- P4: Pipeline optimization (caching, matrix, reusable)

Pipeline Flow:

```
git push
  ↓
Build image
  ↓
Run tests (unit, integration)
  ↓
Security scans (Trivy, tfsec, SonarCloud)
  ↓
Deploy to environment (with approval if needed)
  ↓
Rollback on failure
  ↓
Notify team (Slack)
```

Cost: $0 (GitHub Actions free tier sufficient)

Time saved: Manual deployment 30 min → Automated 5 min

Next: Section 8 - Observability (monitor what CI/CD deploys)

All CICD files available in Repo. You can run cicd and view run.

(all code files in repo)